

PROJECT TITLE: MONTHLY LAND INSTALLMENT PAYMENT SYSTEM.

STUDENT NAME: ATUKWATSE AVIN

REG NO: 2025/BCS/056/PS

COURSE UNIT: DATABASE SYSTEMS

COURSE CODE: CSC 1204

INSTITUTION: MBARARA UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY: COMPUTING AND INFORMATICS

PROGRAM: BACHELOR OF SCIENCE COMPUTER SCIENCE

SUBMISSION DATE: Thursday, 7th May, 2026

MONTHLY LAND INSTALLMENT PAYMENT SYSTEM.

1. Executive Summary

This report presents the development and implementation of a database system for a Monthly Land Installment Payment System. The purpose of this system is to facilitate the efficient tracking and management of monthly land purchase payments made by clients to a real estate or land sales organisation. The system enables staff to record, monitor, and notify clients of their installment obligations, outstanding balances, and payment confirmations.

The database was developed using MySQL, which performs core data management tasks including the insertion, modification, and retrieval of payment records. It incorporates well-structured tables, enforced relationships through foreign keys, and sample queries that demonstrate data retrieval and reporting capabilities.

This report covers the business case, logical and physical database designs, system implementation, table descriptions, sample queries with their results, and a conclusion.

2. Business Case

The purpose of the system is to collect and manage monthly land installment payment data for clients purchasing plots on a deferred payment schedule. This helps land-selling organisations to track payment status, identify overdue accounts, and send timely notifications to clients.

Application domain of the system.

This system supports real estate and land sales businesses by allowing:

- Staff to record each monthly installment payment per client and plot.
- Management to monitor outstanding balances and overdue payments.
- The system to automatically send payment reminders and confirmations to clients.
- Decision-making to identify trends in payment patterns and client reliability.

It has both tangible and intangible benefits;

Tangible Benefits include;

- Efficient recording and retrieval of installment payment data.
- Improved decision-making through structured financial records.
- Accountability tracking for both clients and processing staff.
- Better cash flow management for the land-selling organisation.

Intangible Benefits include;

- Enhanced client trust through timely and accurate payment notifications.
- Improved staff efficiency in payment processing.
- Reduced disputes due to transparent and auditable payment records.

The system has also got costs which are both tangible and intangible;

- Tangible costs: Development and database hosting expenses; staff training and IT support.
- Intangible costs: Resistance to adopting a digital payment tracking system; data privacy concerns regarding client financial information.

User Issues about the system are;

- There is need for basic computer skills to use the system.
- Need for training requirements for both clients and land sales staff.

Risks involved within the system include;

- Unauthorised access to sensitive financial and client data.
- Incorrect data entry leading to inaccurate payment records.
- Technical failures affecting data availability.

Limitations:

- Limited access for clients who are not digitally connected.
- Dependence on staff to enter data accurately and on time.
- No multi-currency support in the current design.
- Language barrier for non-English-speaking clients.

3. Database design based on the requirements.

Logical design

a. List of Entities:

- Client (clients)
- Plot / Parcel (parcels)
- Staff / Agent (agents)
- MonthlyLandInstallment (monthly_land_installments)
- InstallmentNotification (installment_notifications)

b. Entities and Attributes:

- Client: client_id, Name, Phone, Email, National ID, Address
- Plot: parcel_id, Description, Location, Size, Total Value
- Staff: agent_id, Name, Role, Phone, Email
- MonthlyLandInstallment: installment_id, sales_month, period_start, period_end, sale_id, plot_id, client_id, staff_id, installment_no, total_price, installment_amount, amount_paid, balance_remaining, due_date, paid_date, payment_method, status, is_open, notes, created_at, updated_at
- InstallmentNotification: notification_id, installment_id, notification_type, channel, client_id, sent_by, message, is_sent, sent_at, read_at, created_at

c. Entity Relationships:

Client – MonthlyLandInstallment relationship:

A client can have many installment records. This is a one-to-many relationship. Multiplicity is 1:n

Plot – MonthlyLandInstallment relationship:

A plot can be associated with many installment payment records. This is a one-to-many relationship. Multiplicity is 1:n

Staff – MonthlyLandInstallment relationship:

One staff member can process many installments. This is a one-to-many relationship. Multiplicity is 1:n

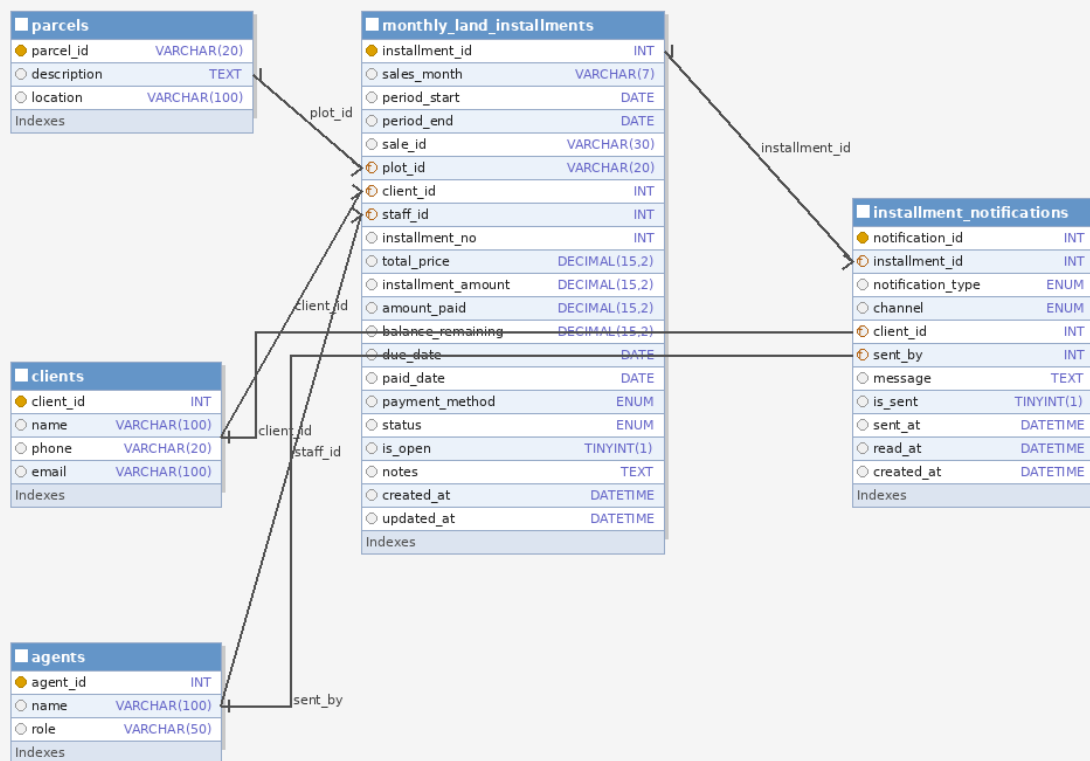
MonthlyLandInstallment – InstallmentNotification relationship:

Each installment can generate many notifications (reminders, confirmations, overdue alerts). This is a one-to-many relationship. Multiplicity is 1:n

Client – InstallmentNotification relationship:

Each notification is directed at one client, while a client can receive many notifications. Multiplicity is n:1

e. A comprehensive ERD for my database.



ERD — Monthly Land Installment Payment System

The Physical Design:

The Database Schema:

Table Name	Attributes	Primary Key	Foreign Keys	Engine
monthly_land_installments	installment_id, sales_month, period_start, period_end, sale_id, plot_id, client_id, staff_id, installment_no, total_price, installment_amount, amount_paid, balance_remaining, due_date, paid_date, payment_method, status, is_open, notes, created_at, updated_at	installment_id	plot_id, client_id, staff_id	InnoDB
installment_notifications	notification_id, installment_id, notification_type, channel, client_id, sent_by, message, is_sent, sent_at, read_at, created_at	notification_id	installment_id, client_id, sent_by	InnoDB

f. User Interface:

This system should be able to allow the user to enter information regarding installment payments, notifications, clients, plots and staff, and then store them.

Below is the interface for the Installment Payments.

The screenshot shows a web application window titled "MONTHLY LAND INSTALLMENTS SYSTEM". It has a navigation bar with tabs: "Installments xx", "Notifications", "Clients", "Plots", and "Staff". The "Installments xx" tab is active. The main form contains the following fields:

- Sales Month (YYYY-MM):
- Period Start:
- Period End:
- Sale ID:
- Plot ID:
- Client ID:
- Staff ID:
- Installment No:
- Total Price (UGX):
- Installment Amount:
- Amount Paid:
- Due Date:
- Payment Method:
- Status:
- Notes:

A "Submit" button is located at the bottom right of the form.

Below is the interface for the Notifications.

The screenshot shows the same web application window, but the "Notifications" tab is active. The main form contains the following fields:

- Installment ID:
- Notification Type:
- Channel:
- Client ID:
- Sent By (Staff ID):
- Message:
- Is Sent:

A "Submit" button is located at the bottom right of the form.

Below is the interface for the Clients.

MONTHLY LAND INSTALLMENTS SYSTEM

Installments

Notifications

Clients ××

Plots

Staff

Client Name:

Phone Number:

Email:

National ID:

Address:

Submit

Below is the interface for the Plots.

MONTHLY LAND INSTALLMENTS SYSTEM

Installments

Notifications

Clients

Plots ××

Staff

Plot ID:

Description:

Location:

Size (Acres):

Total Value (UGX):

Submit

The interface for the Staff.

MONTHLY LAND INSTALLMENTS SYSTEM

Installments

Notifications

Clients

Plots

Staff ××

Staff Name:

Role:

Phone Number:

Email:

Submit

PHYSICAL DESIGN.

The following SQL code is the physical implementation of the Monthly Land Installment Payment System which creates tables for the entities above.

SHOW DATABASES

```
> show databases;
+-----+
| Database |
+-----+
| information_schema |
| monthly_land_installments |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)
```

SHOWING TABLES

```
> show tables;
+-----+
| Tables_in_monthly_land_installments |
+-----+
| installment_notifications |
| monthly_land_installments |
+-----+
2 rows in set (0.00 sec)
```

MONTHLY LAND INSTALLMENTS TABLE

```
CREATE TABLE monthly_land_installments (
  installment_id INT NOT NULL AUTO_INCREMENT,
  sales_month VARCHAR(7) NOT NULL,
  period_start DATE NOT NULL,
  period_end DATE NOT NULL,
  sale_id VARCHAR(30) NOT NULL,
  plot_id VARCHAR(20) NOT NULL,
  client_id INT NOT NULL,
  staff_id INT NOT NULL,
  installment_no INT NOT NULL DEFAULT 1,
  total_price DECIMAL(15,2) NOT NULL DEFAULT 0,
  installment_amount DECIMAL(15,2) NOT NULL DEFAULT 0,
  amount_paid DECIMAL(15,2) NOT NULL DEFAULT 0,
  balance_remaining DECIMAL(15,2) NOT NULL DEFAULT 0,
  due_date DATE NOT NULL,
  payment_method ENUM('Cash','Bank Transfer','Mobile Money','Cheque') NOT NULL,
  status ENUM('Pending','Paid','Overdue','Partially Paid','Cancelled') NOT NULL
  DEFAULT 'Pending',
  is_open TINYINT(1) NOT NULL DEFAULT 1,
  notes TEXT DEFAULT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (installment_id),
  FOREIGN KEY (plot_id) REFERENCES parcels(parcel_id),
  FOREIGN KEY (client_id) REFERENCES clients(client_id),
  FOREIGN KEY (staff_id) REFERENCES agents(agent_id)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
> desc monthly_land_installments;
```

Field	Type	Null	Key	Default	Extra
installment_id	int	NO	PRI	NULL	auto_increment
sales_month	varchar(7)	NO		NULL	
period_start	date	NO		NULL	
period_end	date	NO		NULL	
sale_id	varchar(30)	NO		NULL	
plot_id	varchar(20)	NO	MUL	NULL	
client_id	int	NO	MUL	NULL	
staff_id	int	NO	MUL	NULL	
installment_no	int	NO		1	
total_price	decimal(15,2)	NO		0.00	
installment_amount	decimal(15,2)	NO		0.00	
amount_paid	decimal(15,2)	NO		0.00	
balance_remaining	decimal(15,2)	NO		0.00	
due_date	date	NO		NULL	
paid_date	date	YES		NULL	
payment_method	enum('Cash','Bank Transfer','Mobile Money',...)	NO		Bank Transfer	
status	enum('Pending','Paid','Overdue','Partially Paid')	NO		Pending	
is_open	tinyint(1)	NO		1	
notes	text	YES		NULL	
created_at	datetime	NO		CURRENT_TS	
updated_at	datetime	NO		CURRENT_TS	on update

21 rows in set (0.01 sec)

INSTALLMENT NOTIFICATIONS TABLE

```
CREATE TABLE installment_notifications (
  notification_id INT NOT NULL AUTO_INCREMENT,
  installment_id INT NOT NULL,
  notification_type ENUM('Due Reminder','Overdue Alert','Payment Confirmed','Partial
  Payment','Cancelled') NOT NULL,
  channel ENUM('SMS','Email','WhatsApp','System') NOT NULL DEFAULT 'SMS',
  client_id INT NOT NULL,
  sent_by INT DEFAULT NULL,
  message TEXT NOT NULL,
  is_sent TINYINT(1) NOT NULL DEFAULT 0,
  sent_at DATETIME DEFAULT NULL,
  read_at DATETIME DEFAULT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (notification_id),
  FOREIGN KEY (installment_id) REFERENCES monthly_land_installments(installment_id),
  FOREIGN KEY (client_id) REFERENCES clients(client_id),
  FOREIGN KEY (sent_by) REFERENCES agents(agent_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
> desc installment_notifications;
```

Field	Type	Null	Key	Default	Extra
notification_id	int	NO	PRI	NULL	auto_increment
installment_id	int	NO	MUL	NULL	
notification_type	enum('Due Reminder','Overdue Alert','Payment Confirmed',...)	NO		NULL	
channel	enum('SMS','Email','WhatsApp','System')	NO		SMS	
client_id	int	NO	MUL	NULL	
sent_by	int	YES	MUL	NULL	
message	text	NO		NULL	
is_sent	tinyint(1)	NO		0	
sent_at	datetime	YES		NULL	
read_at	datetime	YES		NULL	
created_at	datetime	NO		CURRENT_TS	TIMESTAMP

11 rows in set (0.00 sec)

IMPLEMENTATION

This includes screenshots of implemented systems, descriptions of tables, queries and their results.

Table descriptions.

```
> desc monthly_land_installments;
```

Field	Type	Null	Key	Default	Extra
installment_id	int	NO	PRI	NULL	auto_increment
sales_month	varchar(7)	NO		NULL	
period_start	date	NO		NULL	
period_end	date	NO		NULL	
sale_id	varchar(30)	NO		NULL	
plot_id	varchar(20)	NO	MUL	NULL	
client_id	int	NO	MUL	NULL	
staff_id	int	NO	MUL	NULL	
installment_no	int	NO		1	
total_price	decimal(15,2)	NO		0.00	
installment_amount	decimal(15,2)	NO		0.00	
amount_paid	decimal(15,2)	NO		0.00	
balance_remaining	decimal(15,2)	NO		0.00	
due_date	date	NO		NULL	
paid_date	date	YES		NULL	
payment_method	enum('Cash','Bank Transfer','Mobile Money',...)	NO		Bank Transfer	
status	enum('Pending','Paid','Overdue','Partially Paid')	NO		Pending	
is_open	tinyint(1)	NO		1	
notes	text	YES		NULL	
created_at	datetime	NO		CURRENT_TS	
updated_at	datetime	NO		CURRENT_TS	on update

21 rows in set (0.01 sec)

```
> desc installment_notifications;
```

Field	Type	Null	Key	Default	Extra
notification_id	int	NO	PRI	NULL	auto_increment
installment_id	int	NO	MUL	NULL	
notification_type	enum('Due Reminder','Overdue Alert','Payment Confirmed',...)	NO		NULL	
channel	enum('SMS','Email','WhatsApp','System')	NO		SMS	
client_id	int	NO	MUL	NULL	
sent_by	int	YES	MUL	NULL	
message	text	NO		NULL	
is_sent	tinyint(1)	NO		0	
sent_at	datetime	YES		NULL	
read_at	datetime	YES		NULL	
created_at	datetime	NO		CURRENT_TS	TIMESTAMP

11 rows in set (0.00 sec)

Sample queries and their results.

```
> select installment_id, sales_month, plot_id, client_id, installment_no,
        total_price, amount_paid, balance_remaining, status from monthly_land_installments;
```

installment_id	sales_month	plot_id	client_id	installment_no	total_price	amount_paid	balance_remaining	status
1	2025-01	PLT-01	1	1	45000000.00	5000000.00	40000000.00	Paid
2	2025-02	PLT-01	1	2	45000000.00	5000000.00	35000000.00	Paid
3	2025-01	PLT-03	2	1	30000000.00	3000000.00	27000000.00	Paid
4	2025-02	PLT-03	2	2	30000000.00	1500000.00	25000000.00	Partially Paid

4 rows in set (0.01 sec)

```
> select * from installment_notifications;
```

notification_id	installment_id	notification_type	channel	client_id	sent_by	message	is_sent
1	1	Payment Confirmed	SMS	1	2	Jan installment of 5M confirmed.	1
2	2	Payment Confirmed	Email	1	2	Feb installment confirmed.	1
3	3	Due Reminder	WhatsApp	2	NULL	Reminder: payment due 28 Feb 2025.	1
4	4	Partial Payment	SMS	2	3	Partial payment of 1.5M received.	1

4 rows in set (0.00 sec)

5. Conclusion.

The Monthly Land Installment Payment System enhances efficiency in land sales organisations by providing a structured and reliable way to record, manage, and track monthly installment payments. The system captures every monthly payment event, maintains an accurate running balance, and supports automatic notification to clients through multiple channels (SMS, Email, WhatsApp). With its well-designed relational schema, enforced data integrity through foreign key constraints, and optimised queries, the system enables quick access to financial data and supports informed decision-making for management.

6. Bibliography

- MySQL Documentation: <https://dev.mysql.com/doc/>
- Connolly, T. & Begg, C. — Database Systems: A Practical Approach to Design, Implementation, and Management, 6th Edition.
- Eclipse Foundation. (2025). Eclipse IDE for Java Developers. <https://www.eclipse.org/>
- Database design tutorials from W3Schools: <https://www.w3schools.com/sql/>